Online Service Migration in Mobile Edge With Incomplete System Information: A Deep Recurrent Actor-Critic Learning Approach

Jin Wang, Jia Hu, Geyong Min, *Member, IEEE*, Qiang Ni, *Senior Member, IEEE*, and Tarek El-Ghazawi, *Fellow, IEEE*

Abstract—Multi-access Edge Computing (MEC) is an emerging computing paradigm that extends cloud computing to the network edge to support resource-intensive applications on mobile devices. As a crucial problem in MEC, service migration needs to decide how to migrate user services for maintaining the Quality-of-Service when users roam between MEC servers with limited coverage and capacity. However, finding an optimal migration policy is intractable due to the dynamic MEC environment and user mobility. Many existing studies make centralized migration decisions based on complete system-level information, which is time-consuming and also lacks desirable scalability. To address these challenges, we propose a novel learning-driven method, which is user-centric and can make effective online migration decisions by utilizing incomplete system-level information. Specifically, the service migration problem is modeled as a Partially Observable Markov Decision Process (POMDP). To solve the POMDP, we design a new encoder network that combines a Long Short-Term Memory (LSTM) and an embedding matrix for effective extraction of hidden information, and further propose a tailored off-policy actor-critic algorithm for efficient training. The extensive experimental results based on real-world mobility traces demonstrate that this new method consistently outperforms both the heuristic and state-of-the-art learning-driven algorithms and can achieve near-optimal results on various MEC scenarios.

Index Terms—Deep reinforcement learning, multi-access edge computing, partial observable markov decision process, service migration

1 Introduction

Recent years have witnessed a booming of emerging mobile applications such as augmented reality, virtual reality, and interactive gaming. These applications require intensive computing power for real-time processing, which often exceeds the limited computing and storage capabilities of mobile devices. To resolve this issue, Multi-access Edge Computing (MEC) [1], a new distributed computing paradigm, was proposed to meet the ever-increasing demands for the Quality-of-Service (QoS) of mobile applications. MEC provides many computing and storage resources at the network edge (close to users), which can effectively cut down the application latency and improve the QoS. Specifically, a mobile application empowered by the MEC consists of a front-end component running on mobile devices, and a back-end service that runs the tasks offloaded from the application on MEC servers [2]. In this way, the MEC enables mobile devices with limited processing power to run complex applications with satisfied QoS.

When considering the user mobility along with the limited coverage of MEC servers, the communications between a mobile user and the user service running on an edge server may go through multiple hops, which would severely affect the QoS. To address this problem, the service could be dynamically migrated to a more suitable MEC server so that the QoS is maintained. Unfortunately, finding an optimal migration policy for such a problem is non-trivial, due to the complex system dynamics and user mobility. Many existing works [3], [4], [5], [6], [7] proposed service migration solutions based on Markov Decision Process (MDP) or Lyapunov optimization under the assumption of knowing the complete system-level information (e.g., workloads of MEC servers, profiles of offloaded tasks, and backhaul network conditions). Thus, they designed centralized controllers (i.e., controllers are placed on edge servers or central cloud) that make migration decisions for mobile users in the MEC system.

The aforementioned methods have two potential drawbacks: 1) in a real-world MEC system, gathering complete system-level information (e.g., the full network states and edge servers' workloads) can bring high communication overheads [8], [9]. 2) the centralized control approach will have scalability issues since its time complexity rapidly increases with the number of mobile users. To address the above issues, some works proposed decentralized service

- *Jin Wang, Jia Hu, and Geyong Min are with the Department of Computer Science, University of Exeter, EX4 4QF Exeter, U.K. E-mail: {jw855, j.hu, g.min}@exeter.ac.uk.*
- *Qiang Ni is with the School of Computing and Communications, InfoLab21, Lancaster University, LA1 4WA Lancaster, U.K. E-mail: q.ni@lancaster.ac.uk.*
- *Tarek El-Ghazawi is with the Department of Electrical and Computer Engineering, The George Washington University, Washington, DC 20052 USA. E-mail: tarek@gwu.edu.*

Manuscript received 14 August 2021; revised 4 April 2022; accepted 3 August 2022. Date of publication 9 August 2022; date of current version 3 October 2023.
This work was supported in part by EU Horizon 2020 INITIATE project under Grant 101008297 and in part by Royal Society International Exchanges under Grant project-IEC\NSFC\211460.
(Corresponding authors: Jia Hu and Geyong Min.)
Digital Object Identifier no. 10.1109/TMC.2022.3197706

1536-1233 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

migration methods based on contextual Multi-Armed Bandit (MAB) [8], [9], [10], [11], where the migration decisions are made by the user side with partially observed information. However, they did not consider the intrinsically large state space and complex dynamics in the MEC system, which may lead to unsatisfactory performance. A recent work [12] modeled the joint optimization problem of service migration and path selection as a partially observable Markov decision process (POMDP) solved by independent Q-learning, which can be unstable and inefficient when handling the MEC environment with continuous state space (e.g., data size, CPU cycle, workload) and complex system dynamics.

To address the aforementioned challenges, we propose a new Deep Recurrent Actor-Critic based service Migration (DRACM) method, which is user-centric and can learn to make online migration decisions with incomplete system-level information based on Deep Reinforcement Learning (DRL). DRL is able to solve complex decision-making problems in various areas, including robotics [13], games [14], networks [15], etc., making it an attractive approach. Distinguished from the existing works, we model the service migration problem as a POMDP with continuous state space and develop a tailored off-policy actor-critic algorithm to efficiently solve the POMDP. The main contributions of this work are listed as follows:

- We model the service migration problem as a Partially Observable Markov Decision Process (POMDP) to capture the intrinsically complex system dynamics in the MEC. We solve the POMDP by proposing a novel off-policy actor-critic method, namely DRACM. Specifically, the distinguishing advantage of this new method is that it is model-free and can quickly learn effective migration policies through end-to-end reinforcement learning (RL), where the agent makes online migration decisions based on the sampled raw data from the MEC environment with minimal human expertise.
- A new encoder network that combines a Long Short-Term Memory (LSTM) and an embedding matrix is designed to effectively extract the hidden information from the sampled histories. Moreover, a tailored off-policy actor-critic algorithm with a clipped surrogate objective function is developed to substantially stabilize the training process and improve the performance.
- We demonstrate how to implement the DRACM efficiently in an emerging MEC framework, where the migration decisions can be made online through the inference of the policy network, while the training of the policy network can be offline, thus reducing the cost of directly interacting with the MEC environment.
- Extensive experiments are conducted to evaluate the performance of the DRACM using real-world mobility traces. The results demonstrate that the DRACM has a stable training process with high adaptivity to different scenarios. Furthermore, it outperforms the online baseline algorithms, and can achieve near-optimal results.

The remainder of this paper is organized as follows. Section 2 gives the problem formulation of service migration. Section 3 presents the DRL backgrounds, POMDP modeling for service migration, details of the DRACM algorithm, and implementation of the DRACM in the emerging MEC system. In Section 4, we evaluate the performance of the DRACM and five baseline algorithms on two real-world mobility traces with various MEC scenarios. We then review the related works in Section 5. Finally, Section 6 concludes this paper.

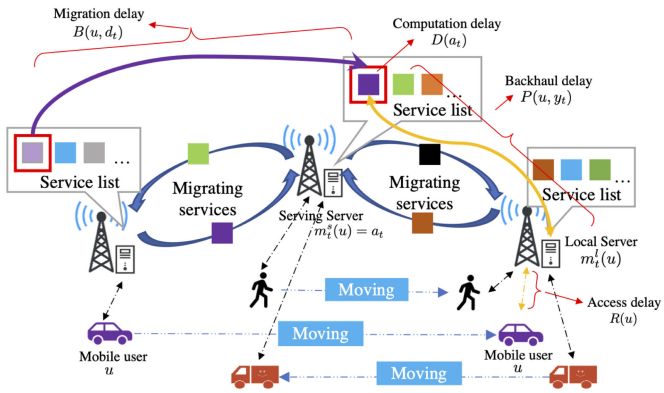


Fig. 1. An example of service migration in MEC.

2 Problem Formulation of Service Migration

As shown in Fig. 1, we consider a heterogeneous multi-user MEC system where a set of U mobile users (indexed by u), $\mathcal{U} = \{1, 2, \ldots, u, \ldots, U\}$, move in a geographical area covered by a set of M MEC servers (indexed by m), $\mathcal{M} = \{1, 2, \ldots, m, \ldots, M\}$, each of which is co-located with a base station. In the MEC system, mobile users can offload their computation tasks to the services provided by MEC servers. We consider a time-slotted model, where a user's location may only change at the beginning of each time slot. The time-slotted model is widely used to address the service migration problem [4], [6], [11], which can be regarded as a sampled version of a continuous-time model. In time slot t ($t = 0, 1, 2, \ldots, T$), all users generate computation tasks according to certain stochastic process. We summarise the main notations of this paper in Table 1.

We define the MEC server that runs the service of a mobile user u at time slot t as the user's *serving server*, denoted as $m_t^s(u)$, and the MEC server that directly connects with the mobile user at time slot t as the user's *local server*, denoted as $m_t^l(u)$. In general, the MEC servers are interconnected via stable backhaul links, thus the mobile user can still access its service via multi-hop communication among MEC servers when it is no longer directly connected to the serving server. However, to maintain satisfactory QoS, the service should be dynamically migrated among the MEC servers as the user moves. In this paper, we use latency as the measurement for the QoS that consists of migration, computation, and communication delays.

When a mobile user changes location, the user makes the migration decision for the current service and then offloads computation tasks to the serving server for processing. Denote the migration decision at time slot t as a_t ($a_t \in \mathcal{M}$) thus $m_t^s(u) = a_t$, where a_t can be any of the MEC servers in this area. In general, the migration, computation, and communication delays are expressed as follows.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Summary of Main Notations

Notation	Description
u, m	Index of the mobile user and the MEC server.
$m_t^s(u)$, $m_t^l(u)$	Index of serving server and local server of the mobile user.
a_t	Migration decision.
$\mathrm{data}_t^s(u)$	Size of the service.
η_t^{m}	Bandwidth of the migration path.
σ_t^{m}	Positive coefficient of the migration delay.
d_t	Hop distance between the current serving server and the previous one.
$\mathrm{data}_t^o(u)$	Data size of the offloaded tasks.
κ	Processing density of the offloaded tasks.
$w_t(a_t)$, $f(a_t)$	Workloads and computing capacity of the serving server.
c_t	Required CPU cycles for processing offloaded tasks.
η_t^{bh}	Bandwidth of the backhaul network.
σ_t^{bh}	Positive coefficient of the backhaul delay.
ρ_t	The upload wireless transmission rate.
ω	Bandwidth of the wireless upload channel.
p_u	Transmission power of the mobile device.
$g_t(u, m_t^l(u))$	Channel gain from the mobile user to the local server.
$\mathcal{N}$	White Gaussian noise.
y_t	Hop distance between the serving server and local server.
$B(u, d_t)$, $D(a_t)$, $E(u, yt)$	Migration delay, computation delay, and communication delay.

Migration Delay. The migration delay is incurred when a service is moved out from the previous serving server. In general, the migration delay is caused by the service interruption time during migration, which increases with the service size and hop distance and involves transmission, propagation, processing, and queuing delays of service data transmission. Formally, the migration delay is a non-decreasing function of d_t [6], [11], [16], which can be defined as:

$$B(u, d_t) = \begin{cases} 0, & \text{if} \quad d_t = 0, \\ \frac{\mathrm{data}_t^s(u)}{\eta_t} + \sigma_t^{\mathrm{m}} d_t, & \text{if} \quad d_t \neq 0, \end{cases} \tag{1}$$

where d_t is the distance between the current serving node $m_t^s(u)$ and the previous one $m_{t-1}^s(u)$, σ_t^{m} is a positive coefficient, η_t^{m} is the network bandwidth along the migration path, $\mathrm{data}_t^s(u)$ is the size of the service that is migrated. Note that, if $d_t = 0$, no migration is occurred, thus the migration delay $B(u, d_t) = 0$.

Computation Delay. At each time slot, the mobile user may offload computation tasks to the serving server for processing. The computing resources of MEC servers are shared by multiple mobile users to process their applications. At time slot t, we denote the total data size of the offloaded task of user u as $\mathrm{data}_t^o(u)$ and the processing density of the offloaded tasks as κ, thus the required CPU cycles for processing the offloaded tasks can be calculated as $c_t = \mathrm{data}_t^o(u)\kappa$. Furthermore, we define the workload of the serving server as $w_t(a_t)$, and the total computing capacity of the serving server as $f(a_t)$. We consider a weighted resource allocation strategy on each MEC server, where tasks are allocated with computation resources proportional to their required CPU cycles. Therefore, the computation delay of running the offloaded tasks at time slot t, can be calculated as

$$D(a_t) = \frac{c_t}{\left(\frac{c_t}{w_t(a_t)+c_t} f(a_t)\right)} = \frac{w_t(a_t) + c_t}{f(a_t)}. \tag{2}$$

Communication Delay. After migrating the service, the communication delay is incurred when the mobile user offloads computation tasks to the serving server. Generally, the communication delay consists of two parts: access delay between the mobile user and the local server, and backhaul delay between the local server and the serving server. The wireless uplink transmission rate from the mobile user to the local server can be define as

$$\rho_t = \omega \log_2\big(1 + SNR(u, m_t^l(u))\big), \tag{3}$$

where ω is the wireless uplink bandwidth and $SNR(u, m_t^l(u))$ is the signal-to-noise ratio (SNR) of the channel between mobile device u and the local server $m_t^l(u))$. The SNR can be formally defined by

$$SNR(u, m_t^l(u)) = \frac{p_u |g_t(u, m_t^l(u))|^2}{\omega \mathcal{N}}, \tag{4}$$

where p_u is the transmission power of the mobile device, $\mathcal{N}$ represents the power spectral density of the white Gaussian noise, $g_t(u, m_t^l(u))$ denotes the channel gain from the mobile user u to the local server $m_t^l(u))$ at time slot t. Specifically, the channel gain can be affected by many factors including the path loss, distance between the user device and the base station, the channel type, etc. Consequently, the value of g_t is difficult to estimate in real-world MEC systems. At time slot t, the access delay can be expressed as

$$R(u) = \frac{\mathrm{data}_t^o(u)}{\rho_t}. \tag{5}$$

The backhaul delay exists when the serving server and local server of the mobile user u are different, i.e., $a_t \neq m_t^l(u)$. The backhaul delay mainly depends on the hop distance along the shortest path and the data size of the offloaded tasks [6], [11], [12]. We denote the hop distance between the serving server a_t and local server $m_t^l(u)$ as $y_t = |a_t - m_t^l(u)|$. Generally, the transmission delay of the computation results can be ignored because of their relative small data size. Therefore, the backhaul delay can be given by

$$P(u, y_t) = \begin{cases} 0, & \text{if} \quad y_t = 0, \\ \frac{\mathrm{data}_t^o(u)}{\eta_t^{\mathrm{bh}}} + \sigma_t^{\mathrm{bh}} y_t, & \text{if} \quad y_t \neq 0. \end{cases} \tag{6}$$

Especially, when the serving server and the mobile user are directly connected ($y_t = 0$), there is no backhaul cost. Overall, the total communication delay at time slot t can be obtained by

$$E(u, y_t) = R(u) + P(u, y_t). \tag{7}$$

Given a finite time horizon T, our objective for the service migration problem is to obtain optimal migration decisions, $\{a_1, a_2, \ldots, a_T\}$, so that the sum of all the above costs (i.e., total delay) is minimal. Formally, the objective is expressed as:

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

$$\min_{a_0,a_1,\ldots,a_T} \sum_{t=0}^{T} B(u,d_t) + D(a_t) + E(u,y_t),$$
$$\text{s.t.} \quad a_t \in \{1,2,\ldots,m,\ldots,M\},$$
$$\forall u \in \{1,2,\ldots,u,\ldots,U\}. \tag{8}$$

Obtaining the optimal solution for the above objective is challenging, which requires knowledge of user mobility and complete system-level information over the entire time horizon. However, in real-world scenarios, it is impractical to gather all the relative information in advance. To address this challenge, we propose a learning-based online service migration method that can make efficient migration decisions based on partially observed information. In the next section, we present our solution in detail.

3 Online Service Migration With Incomplete Information

Service migration in MEC is intrinsically a sequential decision-making problem with a partially observable environment (i.e., with incomplete system information), which can be naturally modeled as a POMDP. We solve the POMDP with the proposed DRACM method to provide effective online migration decisions. Before presenting the details of our solution, we first introduce the necessary backgrounds.

3.1 Backgrounds of RL and POMDP

Reinforcement Learning. RL can solve sequential decision-making problems by learning from interaction with the environment. In general, RL uses the formal framework of MDP, which is defined by a tuple $(\mathcal{S},\mathcal{A},\mathcal{P},\mathcal{R},\gamma)$, to represent the interaction between a learning agent and its environment. Specifically, $\mathcal{S}$ is the state space, $\mathcal{A}$ denotes the action space, $\mathcal{P}$ is the environment dynamics, $\mathcal{R}$ represents the reward function, and γ is the discounted factor. The policy, $\pi(\cdot|s_t)$, represents the distribution over actions given a state s_t. The return from state s_t, which is defined as $G_t(\tau) = \sum_{i=t}^{T} \gamma^{i-t} r_t$, is the sum of discounted rewards along a trajectory $\tau := \{s_0,a_0,r_0,s_1,a_1,r_1,\ldots,s_T,a_T,r_T\}$. The goal of RL is to find an optimal policy π^*, so that the expected return, $\mathbb{E}_{\tau\sim p(\tau|\pi^*)}[G_0(\tau)]$, is maximal.

The action-value function is defined by the expected return after taking an action a_t in state s_t and thereafter following policy π, $q_\pi(s_t,a_t) = \mathbb{E}_\pi[G_t|s_t,a_t]$. An optimal action-value function, which is defined as $q^*(s_t,a_t) = \max_\pi q_\pi(s_t,a_t)$, is the maximum action value achievable by any policy for state s_t and action a_t. The valued-based DRL methods (e.g., deep Q-learning (DQL) [17]) use the deep neural network to approximate the optimal action-value function, $q^*(s_t,a_t;\theta^Q)$ where θ^Q are parameters of the deep neural network. They obtain the optimal policy by greedily selecting the action with maximal action value, where $a_t = \arg\max_a q^*(s_t,a;\theta^Q)$. However, since DQL indirectly obtains a deterministic policy by training the Q-network (i.e., a neural network that is used approximate the action-value function), it generally has a low convergence rate [18]. The large and complex state space of the MEC environment exacerbates this issue. Besides, the training target of DQL is obtained by one-step bootstrapping of the Q-network, which can be a highly biased estimation of the true action values. Introducing bias may harm the convergence of the algorithm, or cause converging to sub-optimal solutions. The above issues make DQL unfit to solve the service migration problem since the learned migration policies may lead to unsatisfied performance. In contrast, the policy-based methods (e.g., asynchronous actor-critic [18] and proximal policy optimization [19]) provide good convergence property for dealing with the complex state space of the environment. They directly parameterized the stochastic policy with a deep neural network rather than using deterministic policy derived from the action-value function. The parameters of the policy network are updated by performing gradient ascent on $\mathbb{E}[G_0(\tau)]$. In this paper, we build our method (i.e., the DRACM) based on the policy-based methods and show the performance comparison between the DQL-based method and the DRACM in Section 4.

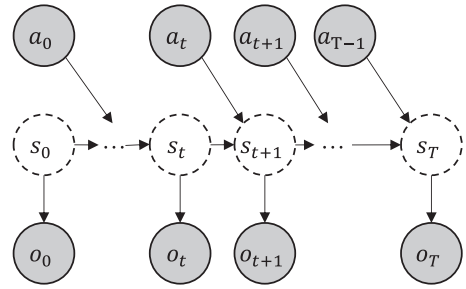


Fig. 2. Graphical model of POMDP.

Partially Observable Markov Decision Process. MDP assumes that states include complete information for decision-making. However, in many real-world scenarios, observing such states is intractable. Therefore, the POMDP, an extension of MDP, is proposed as a general model for the sequential decision-making problem with a partially observable environment, which is defined by a tuple $(\Omega,\mathcal{S},\mathcal{A},\mathcal{P},\mathcal{R},\mathcal{O},\gamma)$. Fig. 2 shows the graphical model of POMDP. Specifically, the state $s_t \in \mathcal{S}$ is latent and the observation $o_t \in \Omega$ contains partial information of the latent state s_t. $\mathcal{O}(o_t|s_t,a_{t-1})$ represents the observation distribution, which gives the probability of observing o_t if action a_{t-1} is performed and the resulting state is s_t. Since the state is latent, the learning agent cannot choose its action directly based on the state. Alternatively, it is possible to inference the filtering distribution given the complete history of its past actions and observations $p(s_t|o_{\leq t},a_{<t}) := b_t$, defined as belief state. Specifically, the history up to time step t is defined by $H_t = \{o_0,a_0,\ldots,o_{t-1},a_{t-1},o_t\}$. If we perform action a_t given the current belief state b_t and get the next observation o_{t+1}, then the next belief state is estimated as follows:

$$b_{t+1} = \frac{\int b_t \mathcal{O}(o_{t+1}|s_{t+1},a_t)\mathcal{P}(s_{t+1}|s_t,a_t)\mathrm{d}s_t}{\int\int b_t \mathcal{O}(o_{t+1}|s_{t+1},a_t)\mathcal{P}(s_{t+1}|s_t,a_t)\mathrm{d}s_t\mathrm{d}s_{t+1}}. \tag{9}$$

However, calculating the above equation requires knowledge of the dynamics model and is computationally expensive [20]. To address this issue, some RL algorithms [20], [21], [22] use approximation methods (e.g., variational inference) to explicitly represent the belief state and sampling latent state from b_t as input to the policy. However, these methods may have high training complexity. Other works [20], [21] assume the latent states as deterministic states, which encode the full history (including a long sequence of

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

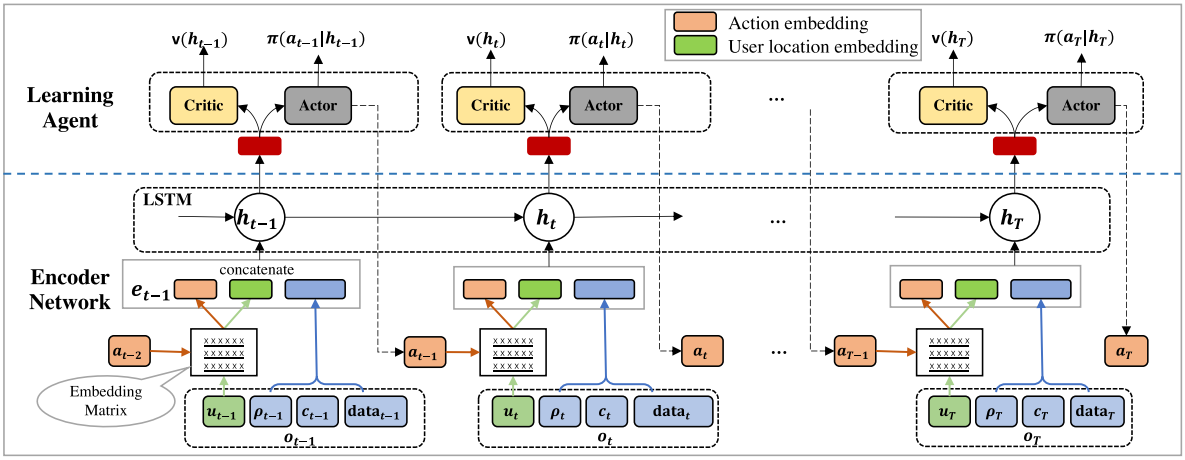


Fig. 3. The architecture of the DRACM for the service migration problem in MEC. The input observation vector includes four components: index of the local server, the current average wireless transmission rate, the required CPU cycles of computation tasks, and the size of transmission data. The output of the actor at each time slot is the probability distribution over the action space (i.e., feasible migration targets).

observations and actions) by LSTM and use the hidden state of LSTM as input to the policy. These methods generally have lower training complexity compared with the former ones. LSTM is an appropriate way for latent information extraction, as it can effectively extract temporal features from long sequential data. Thus, we use the hidden states of LSTM to represent the hidden states of the POMDP. Our algorithm can achieve excellent performance and is much easier to implement in MEC scenarios compared to the methods based on inferring the belief state. In the next subsection, we present the motivations of POMDP modeling for service migration problem and the detailed definition of the model.

3.2 POMDP Modeling for Service Migration Problem

Key factors that affect the migration decision of a mobile user at a time slot are the mobility of the user, the offloading tasks' profile, the workloads of edge servers, and the resource allocations of edge servers, etc. Ideally, the user can make optimal migration decisions if knowing complete information related to the decision-making process. However, some information are hard to obtain for the user side. For example, at each time slot, the workloads of edge servers are determined by the task requests from their associated mobile users and their available computation resources. However, it is unlikely for a specific mobile user to get such information before making migration decisions. To make effective decisions based on partially observable information, POMDP is a natural choice to model the problem, which gives the agent the ability to effectively estimate the outcome of its actions even when it cannot exactly observe the state of its environment. In our POMDP modeling, the mobile user treats the unobserved information (e.g., workloads and resource allocations of MEC servers) as a part of the latent state. Differing from the simplified model such as MAB, POMDP does not ignore the intrinsic large state space and complex dynamics of the service migration problem, thus solving the POMDP can result in more effective decisions.

The detailed POMDP model of service migration is defined as follows:

- *Observation:* The observation contains information that is accessible from the user side, which is defined by a tuple of the local server $m_t^l(u)$, the transmission rate of wireless network ρ_t, the required CPU cycles of computation tasks c_t, and the size of transmission data, $data_t$:

$$o_t := (m_t^l(u), \rho_t, c_t, \text{data}_t^o). \tag{10}$$

  Note that the geographical location of the mobile user is an indirect factor that affects the migration decisions, which determines the local server associated with the mobile user and affects the transmission rate (included in our definition of the observation, Eq. (10)). Therefore, we define the local server $m_t^l(u)$ as a component of the observation rather than the geographical location of the mobile user.
- *Action:* At each time slot, the service can be migrated to any of the MEC servers in the area. Therefore, an action is defined as $a_t \in \mathcal{M}$.
- *Reward:* The reward at each time slot is defined as the negative sum of migration, computation, and communication delays, which is formally expressed as

$$r_t := -[B(u, d_t) + D(a_t) + E(u, y_t)]. \tag{11}$$

Solving the above POMDP is non-trivial due to the complex dynamics and continuous state space of the MEC environment. In the next subsection, we present our method, DRACM, to solve the above POMDP.

3.3 Deep Recurrent Actor-Critic Based Service Migration (DRACM)

Fig. 3 shows the overall architecture of the DRACM, which follows an end-to-end principle with raw history sampled from the environment as input and the migration decisions as output. The DRACM consists of two parts: the encoder network and the learning agent, where the encoder network learns to effectively represent the latent state of the POMDP based on the history and the learning agent learns to make effective migration decisions. The goal of the encoder network is to infer the latent state of the POMDP based on the observed history:

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

$$p(s_{1:T}|o_{1:T}, a_{1:T-1}) = \prod_{t=1}^{T} p(s_t|s_{t-1}, a_{t-1}, o_t). \tag{12}$$

Here, we include a LSTM to approximate the above function where the hidden state of the LSTM, h_t, is used to represent the latent state s_t of the POMDP, thus we have

$$h_t = f_{\text{enc}}([o_{\leq t}, a_{<t}]; \theta) = f_{\text{enc}}([o_t, a_{t-1}], h_{t-1}; \theta), \tag{13}$$

where $t \in [1, T]$, f_{enc} and θ represent the inner process and parameters of the encoder network, respectively.

To improve the representation ability of the features u_t and a_{t-1}, we convert them into embeddings by looking up a trainable $|\mathcal{M}| \times d_e$ matrix, where d_e is the dimension of embedding vectors. Subsequently, the action embedding, user location embedding, and the rest components of the observation are concatenated as a vector, e_t, feeding into the LSTM to produce the hidden state h_t.

The learning agent is based on a standard actor-critic structure. Both actor and critic are parametrized by neural networks with the hidden state h_t as input. We denote ϕ and ψ as the parameters of actor and critic networks, respectively. The actor network aims at approximating the policy, $\pi(a_t|h_t; \phi)$, which outputs a distribution over the action space at time step t given h_t. Meanwhile, the critic network, $v(h_t, \psi)$, approximates the value function that is an estimation of the expected return when starting in h_t and following the policy π thereafter.

Denote the trajectory sampled from the environment following policy π as $\tau = \{o_0, a_0, r_0, \ldots, o_T, a_T, r_T\}$. The critic network can be updated by minimizing the mean square error of one-step temporal differences δ_t based on the sampled trajectories, which is formally defined as

$$L^{\text{critic}}(\psi, \theta) = \mathbb{E}_{\tau \sim p(\tau|\pi)} \left[\sum_{t=0}^{T} \delta_t^2 \right], \tag{14}$$

$$\delta_t = r + \gamma v(h_{t+1}; \psi) - v(h_t; \psi), \tag{15}$$

where the h_t can be obtained by Eq. (13). The objective of the actor is to find an optimal policy that maximizes the accumulated reward, which can be formally expressed as

$$L^{\text{act}}(\phi, \theta) = \mathbb{E}_{\tau \sim p(\tau|\pi)} \left[\sum_{t=0}^{T} \gamma^t r_t \right]. \tag{16}$$

The optimal policy can then be obtained by gradient assent through policy gradient with one-step actor-critic [23], where the gradient of the above objective function can be calculated by

$$\nabla_{\theta,\phi} L^{\text{act}} = \mathbb{E}_{\tau \sim p(\tau|\pi)} \left[\sum_{t=0}^{T} \delta_t \nabla_{\theta,\phi} \log \pi(a_t|h_t; \phi) \right]. \tag{17}$$

However, directly applying the above on-policy (i.e., using the same policy for training and sampling) objective has some drawbacks when solving the service migration problem. First, we cannot train the policy network offline with mini-batches by using on-policy objective. This can lead to severe sample efficiency problem, since the learning agent needs to resample trajectories from the environment after each gradient update. Especially, in the MEC system, frequently interacting with the environment to get the training samples is costly. Second, the on-policy objective has limited exploring ability, thus the policy can easily get stuck in a local optima. Third, to reduce the variance of the objective function, Eq. (17) includes a biased estimator δ_t. However, introducing bias may harm the convergence of the algorithm.

Algorithm 1. Deep Recurrent Actor-Critic Based Service Migration (DRACM)

Initialize the parameters of behavior policy ϕ', behavior encoder network θ', target policy ϕ, target encoder network θ, and critic network ψ,

for $k = 0, 1, 2, \ldots, n$ **do**
  ▷ % Start sampling process %
  Synchronize the parameters: $\theta' \leftarrow \theta$, $\phi' \leftarrow \phi$.
  Sample a set of trajectories $D_\tau = \{\tau_0, \tau_1, \ldots \tau_n\}$ by running the behavior policy $\pi'(a_t|h'_t; \phi')$ in the environment, where $h'_t = f_{\text{enc}}([o_{\leq t}, a_{<t}]; \theta')$.
  Compute the advantage estimator, $\hat{A}_t$, according to Eq. (21).
  ▷ % Start target policy updating process %
  for $j = 0, 1, 2, \ldots, m$ **do**
    Update the parameters of encoder network θ, target policy network ϕ, and critic network ψ,
    $\theta \leftarrow \theta + \nabla_\theta L_{\text{c}}^{\text{act}}(\phi, \theta) - \nabla_\theta L^{\text{critic}}(\psi, \theta)$,
    $\phi \leftarrow \phi + \nabla_\phi L_{\text{c}}^{\text{act}}(\phi, \theta)$,
    $\psi \leftarrow \psi - \nabla_\psi L^{\text{critic}}(\psi, \theta)$,
    by mini-batch gradient updates based on collected trajectories D_τ with *Adam*.
  end for
end for

To improve the sample efficiency, an appropriate way is to apply off-policy (i.e., training a policy different from that was used to sample the data) algorithm that can train the policy with mini-batches and reduce the interaction frequency with the environment. Formally, the off-policy target can be expressed by:

$$L^{\text{act}}(\phi, \theta) = \mathbb{E}_{\tau \sim p(\tau|\pi')} \left[\sum_{t=0}^{T} \frac{\pi_t}{\pi'_t} \gamma^t r_t \right], \tag{18}$$

where $\pi'(a_t|h'_t; \phi')$ is the behavior policy for sampling trajectories, which does not participate in gradient updates. $\pi(a_t|h_t; \phi)$ is the target policy for optimization. $\frac{\pi_t}{\pi'_t}$ is the importance sampling ratio which is used to correct the distribution errors caused by the difference between the behavior and target policies. However, directly training the above off-policy target can be unstable and even divergent [24]. To stabilize the training, inspired by the previous works on RL [19], [25], [26], we introduce an off-policy training method with a surrogate objective function:

$$L_{\text{c}}^{\text{act}}(\phi, \theta) = \mathbb{E}_{\tau \sim p(\tau|\pi')} \left[\sum_{t=0}^{T} g_{\text{clip}}(\pi'_t, \pi_t, \hat{A}_t) + c_h \mathcal{H}(\pi_t) \right], \tag{19}$$

$$g_{\text{clip}}(\pi'_t, \pi_t, \hat{A}_t) = \min\left(\frac{\pi_t}{\pi'_t} \hat{A}_t, \text{clip}_{1-\epsilon}^{1+\epsilon}\left(\frac{\pi_t}{\pi'_t} \right) \hat{A}_t \right), \tag{20}$$

$$\hat{A}_t(h_t; \psi) = \sum_{l=0}^{T} (\gamma\lambda)^l \delta_{t+l}. \tag{21}$$

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

Here, the clip function, $\text{clip}_{1-\epsilon}^{1+\epsilon}$, is used to limit the value of the importance sampling ratio by removing the incentive for moving the ratio outside of the interval $[1-\epsilon, 1+\epsilon]$, thus it can prevent very large policy updates and stabilize the training. $c_h\mathcal{H}(\pi_t)$ is a regularization term to further encourage exploration during training, where $\mathcal{H}(\pi_t)$ denotes the entropy of the policy and c_h is a coefficient. To balance the trade-off between variance and bias of the training objective, we utilize the generalized advantage estimator (GAE) [26], $\hat{A}_t$, as given by Eq. (21), where $\lambda \in [0, 1]$ is used to control the trade-off between bias and variance. GAE can dramatically reduce the variance of the objective while keeping a tolerable bias level.

Algorithm 1 summarizes the training process of the DRACM. Each training loop consists of the sampling process and the target policy updating process. In the sampling process, we first synchronize the parameters of the behavior and target networks (include policy network and encoder network), and then sample a set of trajectories from the environment using the behavior encoder and policy networks. The advantage estimator, $\hat{A}_t$, can then be obtained based on the sampled trajectories. Next, in the target policy updating process, we conduct training of m loops to update the parameters of the encoder network, policy network, and critic network via mini-batch stochastic gradient descent with *Adam* [27]. After training, the target policy and encoder networks can be deployed to the end device for making online migration decisions by neural network inference. Specifically, we obtain the sequence of migration decisions through LSTM inference which has a linear time complexity of $O(n)$ [28], where n is the number of time slots (i.e., the input sequence length of the LSTM). In the next subsection, we discuss how to implement the DRACM in the emerging MEC system.

3.4 The DRACM Empowered MEC Framework

The emerging MEC system defined by ETSI consists of three levels: user level, edge level, and remote level [1]. The user level includes various mobile devices such as smartphones and vehicles. The edge level consists of multiple edge servers where each server provides services for processing tasks that are offloaded by mobile users. The edge servers are connected through backhaul links, thus the service can be migrated among them. The remote level includes data centers with large storage and computing capacity. Fig. 4 shows the overall framework of integrating the DRACM into the three-level MEC system. Four key components (*experience collector*, *migration decision maker*, *experience pool*, and *target policy trainer*) of the DRACM are deployed at the user and remote level:

- At the user level, the *experience collector* is responsible of collecting the information of observations and rewards from the MEC environment (Step ①). It sends the history $H_t = \{o_0, a_0, \ldots, a_{t-1}, o_t\}$ to the *migration decision maker* for online decision-making (Step ②), and the collected trajectories to the *experience pool* for the target policy training (Step ④). The *migration decision maker* includes behavior policy and encoder networks. It downloads parameters from the *target policy trainer* as the initial values of the behavior policy and encoder networks (Step ⑤), and decides the migration actions based on the observed history (Step ③).
- At the remote level, the *experience pool* stores the sampled trajectories from mobile users. The *target policy trainer* is in charge of training the target policy based on the sampled trajectories.

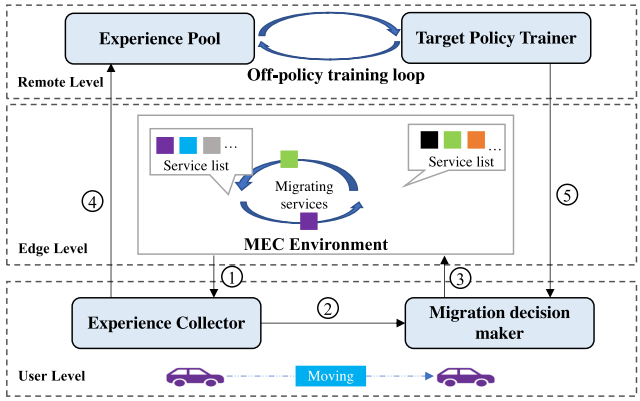


Fig. 4. The framework of DRACM empowered MEC system. The data flows in this framework are: ① the observation o_t and reward r_t from the MEC environment, ② the history $H_t = \{o_0, a_0, \ldots, a_{t-1}, o_t\}$ for migration decision-making, ③ the migration action, a_t, made by the behavior policy, ④ the collected trajectories uploaded to the experience pool, ⑤ the parameters of the trained target policy and encoder networks for service migration.

According to Algorithm 1, the *target policy trainer* conducts multiple training loops with mini-batch gradient updates based on the collected trajectories in the *experience pool*. Note that the training can be offline without directly interacting with the MEC environment. After training, the *target policy trainer* sends the updated parameters of policy and encoder networks to mobile users for the next-round of sampling process.

Overall, our proposed system framework enables an effective offline training process of the policy network and distributed online decision-making for service migration via the inference of the trained policy network. To effectively extract the hidden information from the observation history, we develop a new encoder network that incorporates LSTM and an embedding matrix for local information. In addition, a tailored off-policy actor-critic algorithm with a clipped surrogate target is designed to improve the sample efficiency and stability during training.

4 Experiments

In this section, we present the comprehensive evaluation results of the DRACM in detail. Our experiments demonstrate that: 1) the DRACM has a stable and efficient training process; 2) the DRACM can autonomously adapt to different MEC scenarios including various user's task arriving rates, applications' processing densities, and coefficients of migration delay. We first introduce the experiment settings based on a real-world MEC environment. Next, we present the baseline algorithms for comparison. Finally, we evaluate the performance of the DRACM and baseline algorithms in different MEC scenarios.

4.1 Experiment Settings

We evaluate the DRACM with two real-world mobility traces of cabs in Rome, Italy [29] and San Francisco, USA [30].

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

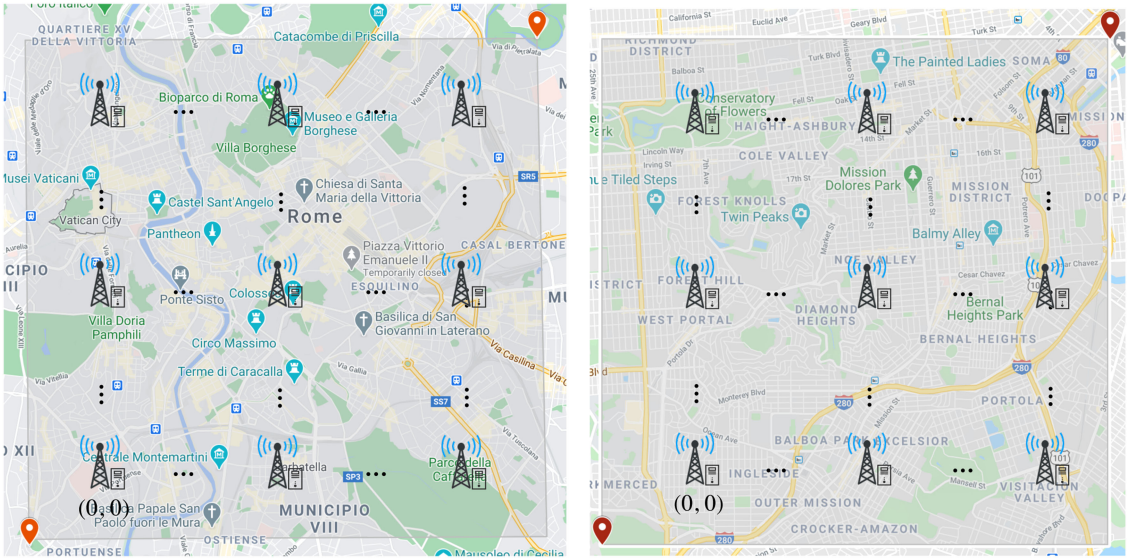


Fig. 5. The central areas of Rome, Italy (8 km $\times$ 8 km area bounded by the coordinate pairs [41.856, 12.442] and [41.928, 12.5387]) and San Francisco (8 km $\times$ 8 km area bounded by the coordinates pairs [37.709, -122.483] and [37.781, -122.391]).

Specifically, we focus our analysis to the central parts of Rome and San Franscisco, as shown in Fig. 5. We consider that 64 MEC servers are deployed in each area, where each MEC server covers a 1 km $\times$ 1 km grid with a computation capacity $f = 128$ GHz (i.e., four 16-core servers with 2 GHz for each core). According to [31], the upload rate of real-world commercial 5 G networks is generally less than 60 Mbps. Therefore, in our environment, the upload rate ρ_t in each grid is set as 60, 48, 36, 24, and 12 Mbps from a proximal end to a distal end. The hop distances between two MEC servers are calculated by Manhattan distance. The location of an MEC server is represented by a 2-D vector (i, j) with respect to a reference location at $(0, 0)$. To calculate the propagation latency, we set the bandwidth of backhaul network, η_t, as 500 Mbps [32] and the coefficient of backhaul delay, σ_t^{bh}, as 0.02 s/hop [12]. The migration delay varies with various service sizes and network conditions, e.g., the migration delay of Busybox (a type of service) ranges from 2.4 to 3.3 seconds [32] with different bachkhaul network conditions. Following some related work on MEC [4], [7], [11], [32], we assume the service size is uniformly distributed in $[0.5, 100]$ MB and the coefficient of migration delay σ_t^{m} is uniformly distributed in $[1.0, 3.0]$ s/hop during our training.

At each time slot, the tasks arriving at a mobile user and those arriving at an MEC server are sampled from Poisson distributions with rates λ_p^u and λ_p^s, respectively. In our experiments, we show the performance of the DRACM under different task arriving rates of mobile users. According to the current works [33], [34], [35], the data size of an offloaded task in real-world mobile applications often varies from 50 KB (sensor data) [33] to 5 MB (image data) [34]. Hence, we set the data size of each offloaded task uniformly distributed in $[0.05, 5]$ MB. Besides, the size of the migration service generally varies from 0.5 MB to 100 MB [7], [32]. Therefore, we uniformly sample the service size from $[0.5, 100]$ MB. The required CPU cycles of each task can be calculated by the product of the data size and processing density, κ, which is uniformly distributed in $[200, 10000]$ cycles/bit, covering a wide range of tasks from low to high computation complexity [36]. We summarize the parameter settings of our simulation environment in Table 2.

4.2 Baseline Algorithms

We compare the performance of the DRACM to that of six baseline algorithms:

- *Always migrate (AM):* A mobile user always selects the nearest MEC server to migrate at each time slot.
- *Never migrate (NM):* The service is placed on an MEC server and never migrate during the time horizon.
- *Multi-armed Bandit with Thompson Sampling (MABTS)*: Some exiting works [10], [11] solve the service migration problem based on MAB. According to the work [11], MABTS uses a diagonal Gaussian distribution to approximate the posterior of the cost for each arm and applies Thompson sampling to handle the trade-off between exploring and exploiting.
- *DQL-based migration (DQLM):* Some recent works [4], [5], [12] adapt DQL to tackle the service migration problem. We directly use a fully connected layer to approximate the action value function that takes the observation as input. Moreover, we use ϵ-greedy to control the exploring-exploiting trade-off as the above works do.

TABLE 2
Parameters of the Simulated Environment

Parameter	Value
Number of MEC servers, M	64
Computation capacity of an MEC server, f	128 GHz
Upload rate of wireless network, ρ_t	{60, 48, 36, 24, 12} Mbps
Bandwidth of backhaul network, η_t	500 Mbps
Coefficient of backhaul delay, σ_t^{bh}	0.02 s/hop
Coefficient of migration delay, σ_t^{m}	$U[1.0, 3.0]$ s/hop
Data size of the service $\mathrm{data}_t^s(u)$,	$U[0.5, 100]$ MB
Data size of each offloaded task $\mathrm{data}_t^o(u)$	$U[0.05, 5]$ MB
Processing density of an offloaded task, κ	$U[200, 10000]$ cycles/bit
User's task arriving rate λ_p^u	2 tasks/slot
MEC server's task arriving rate λ_p^s	$U[5, 20]$ tasks/slot

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
Hyperparameters of the DRACM

Hyperparameter	Value	Hyperparameter	Value
LSTM Hidd. Units	256	Embedding Dim. d_e	2
Actor Layer Type	Dense	Actor Hidd. Units	128
Critic Layer Type	Dense	Critic Hidd. Units	128
Learning Rate	0.0005	Optimizer	Adam
Discount λ	0.95	Discount γ	0.99
Coefficient c_h	0.01	Clipping Value ϵ	0.2

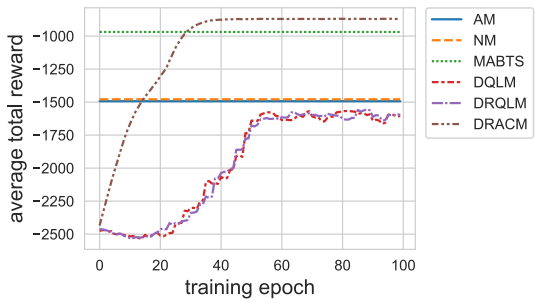


Fig. 6. Average total reward of the DRACM and baseline algorithms with the mobility traces of Rome.

- *Deep Recurrent Q-learning based migration (DRQLM):* Deep recurrent Q-learning is a variant DQL method that can be used to solve POMDP. For fair comparisons, we use a similar neural network structure as DRACM to approximate the action-value function for DRQLM, but use the objective function of the DQL-based method as the training target.
- *Optimal migrate (OPTIM):* Assuming the user mobility trace and the complete system-level information over the time horizon are known ahead, the service migration problem can be transformed to the shortest-path problem [3], [4], which can be solved by the Dijkstra algorithm.

The NM, AM, MABTS, DQLM, DRQLM, and DRACM algorithms can run online, while the OPTIM is an offline algorithm which defines the performance upper-bound of service migration algorithms.

4.3 Evaluation of the DRACM and Baseline Algorithms

We first evaluate the training performance of the DQLM, DRQLM, and DRACM on two different mobility trace datasets [29], [30]. Each training dataset includes 100 randomly picked mobility traces, where each trace has 100 time slots of three-minute length each. Table 3 lists the hyperparameters in training. The neural network structure of the DRQLM is similar to the DRACM with the same encoder network. The difference is that, rather than using the actor-critic structure, the DRQLM is based on the Q-network that includes a fully connected layer with 128 hidden units to approximate the action-value function and chooses the action with the largest action-value at each time step. DQLM directly use Q-network to produce the state-action values without using the encoder network. We train the DQLM, DRQLM, and DRACM with the same learning rate, mini-batch size, and number of gradient update steps.

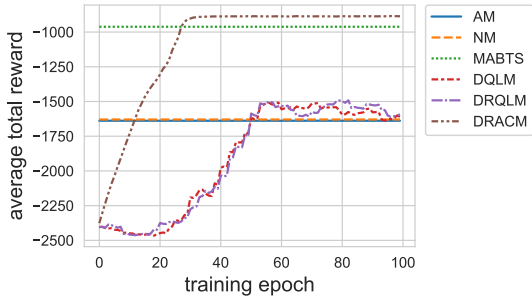


Fig. 7. Average total reward of the DRACM and baseline algorithms with the mobility traces of San Francisco.

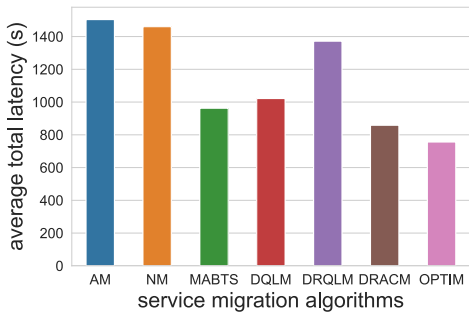


Fig. 8. Average total latency (s) of service migration over the time horizon (250 minutes) on the testing dataset from mobility traces of Rome.

Figs. 6 and 7 show the training results of DRACM, DRQLM, and DQLM on mobility traces of Rome and San Francisco, respectively. The other baseline algorithms do not involve the training process for neural networks, thus we show their final performance. The network parameters of DRACM, DRQLM, and DQLM are initialized by random values, thus they randomly select actions to explore the environment and achieve the worst results compared to other baseline algorithms before training. However, the DRACM quickly surpasses NM and AM after 12 epochs and keeps growing on both mobility traces. After 25 training epochs, the average total reward of the DRACM remains stable, which shows the excellent convergence property of the DRACM. Besides, the final stable results of the DRACM on both mobility traces beat all baseline algorithms.

To evaluate the generalization ability of the DRACM, we test the trained target policy on testing datasets of both mobility traces, where each test dataset includes 30 randomly picked mobility traces that were not included in the training dataset. We evaluate the testing time of all online algorithms including MABTS, DQLM, DRQLM, and DRACM on our laptop with a CPU (2.9 GHz 6-Core Intel Core i7), where the average testing time is 0.332 s, 0.345 s, 0.325 s, and 0.328 s, respectively. We do not evaluate the testing time of AM, NM, and RM, since the time complexity of these simple algorithms is O(1). Figs. 8 and 9 present the results of the average total latency of DRACM and baseline algorithms on Rome and San Francisco mobility traces, respectively. We found the DRACM achieves the best performance compared to online baseline algorithms on both mobility traces.

Specifically, Fig. 8 shows that the DRACM outperforms the DQLM, DRQLM and MABTS by 16%, 37%, and 11%, respectively. Fig. 9 indicates that the DRACM surpasses the

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

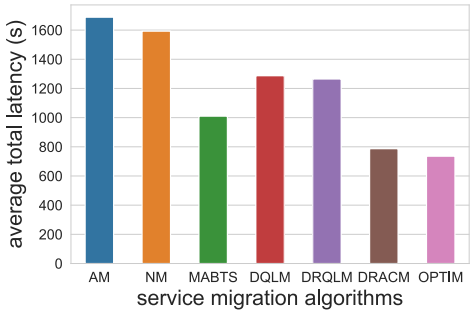


Fig. 9. Average total latency (s) of service migration over the time horizon (250 minutes) on the testing dataset from mobility traces of San Francisco.

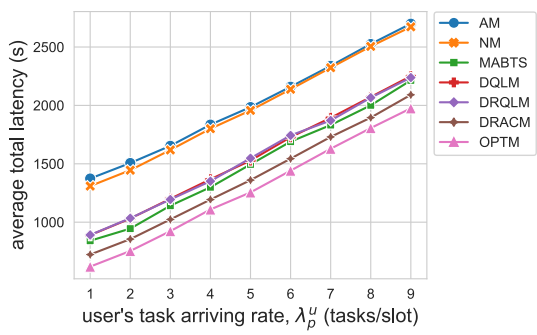


Fig. 10. Average total latency (s) of service migration over the time horizon (250 minutes) with different task arriving rates of users (mobility traces of Rome).

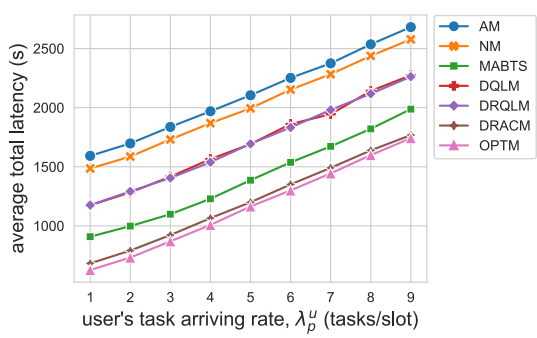


Fig. 11. Average total latency (s) of service migration over the time horizon (250 minutes) with different task arriving rates of users (mobility traces of San Francisco).

DQLM, DRQLM and MABTS by 39%, 38%, and 22%, respectively. Furthermore, the DRACM achieves near-optimal results within 12% of the optimum on both mobility traces.

We then test the DRACM and baseline algorithms with different task arriving rates of users on both mobility traces. As shown in Figs. 10 and 11, the average total latencies of all evaluated algorithms increase with the rise of user's task arriving rate, since the average number of offloaded tasks increases at each time slot. The evaluation results show that the DRACM adapts well among different task arriving rates of users, where it outperforms the DQLM, DRQLM, and MABTS in the average total latency by up to 42%, 42%, and 24%, respectively. Moreover, in all cases, the results of DRACM are close to the optimal values.

Next, we investigate the performance of the DRACM with different processing densities. For a real-world mobile

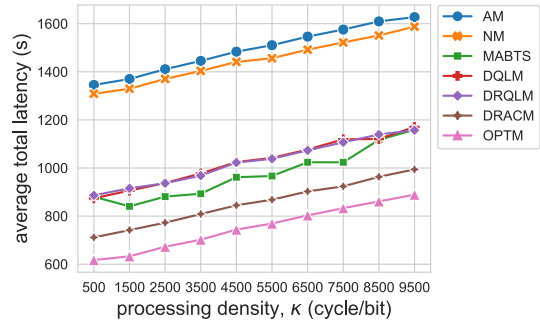


Fig. 12. Average total latency (s) of service migration over the time horizon (250 minutes) with different processing densities (mobility traces of Rome).

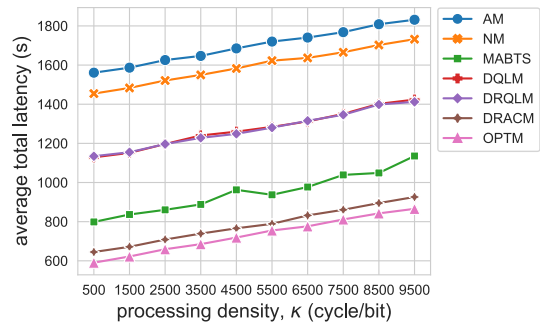


Fig. 13. Average total latency (s) of service migration over the time horizon (250 minutes) with different processing densities (mobility traces of San Francisco).

application, the higher is the processing density, the more computation power is required for processing the application. Figs. 12 and 13 depict the average total latency of DRACM on Rome mobility traces and San Francisco mobility traces, respectively. We find that the DRACM adapts well to the change of processing density on both mobility traces, where it outperforms all online baselines.

Migration delay is another important factor that influences the overall latency. To investigate the impact of the migration delay, we evaluate the DRACM and baseline algorithms on the testing datasets with different coefficients of migration delay. Intuitively, when the migration delay is high, a mobile user may not choose to frequently migrate services. As shown in Figs. 14 and 15, the NM algorithm keeps the similar performance in all cases while the performance of other algorithms drops with the increase of m_c^t. This is because that the NM does not involve the migration process and thus has no migration delay. In Fig. 14, we find the MABTS suffers serious performance degradation as m_t^c increases. When the m_c^t is low (e.g., $m_c^t = 1.0$), the MABTS achieves similar results as the DRACM. However, when $m_c^t > 4$, the performance of MABTS becomes even worse than the DQLM and DRQLM. Compared to RL-based methods like the DQLM and DRACM, MABTS is "short-sighted" since it only considers the one-step reward rather than explicitly optimizes the total reward over the entire time horizon. Overall, the DRACM autonomously learns to adapt among the scenarios with different migration delays, which achieves the best performance compared to the online baselines (with up to 31% improvement over the MABTS and up to 40% improvement over both DQLM and DRQLM), and obtains near-optimal results in our experiments.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

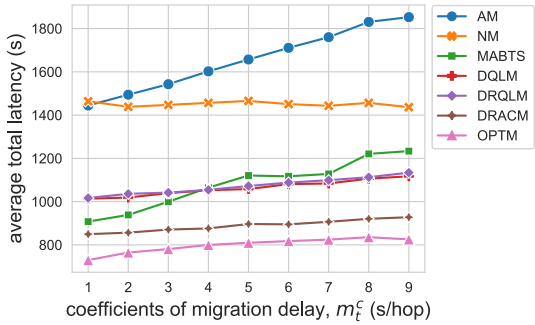


Fig. 14. Average total latency (s) of service migration over the time horizon (250 minutes) with different coefficients of migration delay (mobility traces of Rome).

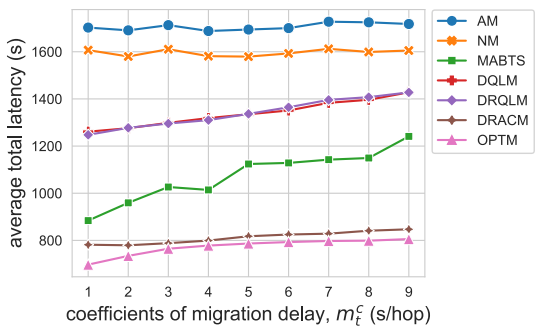


Fig. 15. Average total latency (s) of service migration over the time horizon (250 minutes) with different coefficients of migration delay (mobility traces of San Francisco).

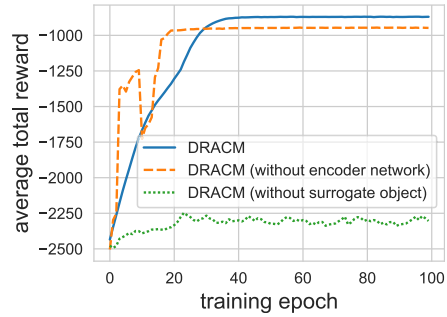


Fig. 16. Average total reward of the DRACM with the mobility traces of Rome.

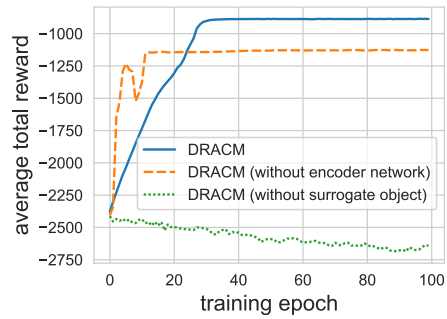


Fig. 17. Average total reward of the DRACM with the mobility traces of San.

4.4 Ablation Study of the DRACM

As the encoder network and the surrogate objective function play important roles in DRACM, we conduct ablation study two show how these parts affect the performance of the DRACM. To investigate the impact of the encoder network, we replace the encoder network of DRACM with a fully connect layer with 256 hidden units, which directly takes the observation as input. To show the impact of the surrogate object, we directly use Eq. (16) as the training objective. The training results are shown in Figs. 16 and 17 on Rome and San Francisco mobility traces, respectively. We found that if we remove the encoder network, the DRACM converges to a local optima and achieves similar but worse performance. Moreover, if we remove the surrogate object, the training of DRACM is divergent on both mobility traces, showing the importance of the surrogate objective function.

The DRACM method has many advantages: 1) the learning-based nature of the DRACM makes it flexible among different scenarios with few human expertise; 2) the user-centric design is scalable for the increasing number of mobile users, where each mobile user makes effective online migration decisions based on the incomplete system information; 3) the tailored off-policy training objective improves both performance and stability of the training process; 4) the design of online decision-making and offline policy training makes the DRACM more practical in real-world MEC systems. Despite the above advantages of the DRACM algorithm, it still has several limitations. For example, when handling rapidly changing dynamics of the MEC environment, the trained policy may not adapt well and requires retraining. To improve the adaptability of DRL methods for the rapidly changing environments, meta reinforcement learning (MRL) is a potential approach that can learn to fast adapt to unseen environments by leveraging previous experiences [37]. In our future work, we will investigate the feasibility to apply MRL to solve the service migration problem with rapidly changing MEC environments. Beyond the scope of service migration, the framework of the DRACM has the potential to be applied to solve more decision-making problems in MEC systems such as task offloading and resource allocation [38].

5 Related Work

Service migration in MEC has attracted intensive research interests in recent years. Rejiba et al. [2] published a comprehensive survey on mobility-induced service migration in fog, edge, and related computing paradigms. We roughly classify the related work into centralized control approach (the central cloud or MEC servers make service migration decisions for all mobile users) and decentralized control approach (each mobile user makes its own migration decisions).

Centralized Control Approach. plenty of works focused on making centralized migration decisions (i.e., the migration decisions are made by ether central cloud or edge servers) based on the complete system-level information to minimize the total cost. Ouyang et al. [3] converted the service migration problem as an online queue stability control problem and applied Lyapunov optimization to solve it. Zhou et al. [7] aimed at solving service migration problem for multi-user dense cellular networks. They developed an energy-efficient online migration method based on the Lyapunov and particle swarm optimizations. Liu et al. [39] propose a multi-agent RL based method for the service migration where

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

agents represent the controllers of MEC servers. Xu et al. [40] formulated the service migration problem as a multi-objective optimization framework and proposed a method to achieve a weak Pareto optimal solution. Wang et al. [6] formulated the service migration problem as a finite-state MDP and proposed an approximation of the underlying state space. They solve the finite-state MDP by using a modified policy-iteration algorithm. There are other recent works that tackled the service migration problem based on RL. Wang et al. [4] proposed a Q-learning based micro-service migration algorithm in mobile edge computing. Wu et al. [5] considered jointly optimizing the task offloading and service migration, and proposed a Q-learning based method combing the predicted user mobility. Addad [41] et al. proposed an enhanced network function virtualization edge computing architecture that incorporates Q-learning based methods to implement service and slice migration. These works considered the case where the decision-making agent knows the complete system-level information. However, in a practical MEC system, collecting complete system-level information can be difficult and time-consuming. Moreover, they adopted the centralized control approach that may suffer from the scalability issue when facing a rapidly increasing number of mobile users.

Decentralized Control Approach. some studies proposed to make migration decisions by the user side based on incomplete system-level information. Ouyang et al. [11] formulated the service migration problem as an MAB and proposed a Thompson-sampling based algorithm that explores the dynamic MEC environment to make adaptive service migration decisions. Sun et al. [9] proposed an MAB based service placement framework for vehicle cloud computing, which can enable the vehicle to learn to select effective neighboring vehicles for its service. Sun et al. [10] developed a user-centric service migration framework using MAB and Lyapunov optimization to minimize the latency with constraints of energy consumption. These methods simplify the system dynamics by modeling with MAB, which ignores the inherently large state space and complex transitions among states in a real-world MEC system. Distinguished from the above works, our method models the service migration problem as a POMDP that has a continuous state space and models complex transitions between states. Moreover, our method is model-free and adaptive to different scenarios, which can learn to make online service migration decisions with minimal expert knowledge. More recently, Yuan et al. [12] investigated the joint service migration and mobility optimization problem for vehicular edge computing. They modeled the MEC environment as a POMDP and proposed a multi-agent DRL method based on independent Q-learning to learn the policy. However, using Q-learning based method to solve the environment with complex dynamics and continuous state space can be unstable and inefficient. Our evaluation results show that our method can achieve stabler training and better results than the DQL-based method.

6 Conclusion

In this paper, we proposed the DRACM, a new method for solving the service migration problem in MEC given incomplete system-level information. Our method is completely model-free and can learn to make online migration decisions through end-to-end RL training with minimal human expertise. Specifically, the service migration problem in MEC is modeled as a POMDP. To solve the POMDP, we designed an encoder network that combines an LSTM and an embedding matrix to effectively extract hidden information from sampled histories. Besides, we proposed a new tailored off-policy actor-critic algorithm with a clipped surrogate objective to improve the training performance. We demonstrated the implementation of the DRACM in the emerging MEC framework, where migration decisions can be made online from the user side and the training for the policy can be offline without directly interacting with the environment. We evaluated the DRACM and five online baseline algorithms with real-world datasets and demonstrated that our DRACM consistently outperforms the online baselines and achieves near-optimal results on a diverse set of scenarios.

Acknowledgments

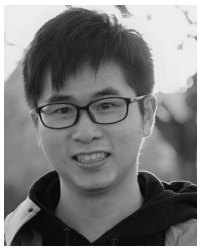
The European Union Commission is not responsible for any use that may be made of the information it contains.

References

[1] D. Sabella et al., "Developing software for multi-access edge computing," *ETSI White Paper*, vol. 20, pp. 1–38, 2019.
[2] Z. Rejiba, X. Masip-Bruin, and E. Marín-Tordera, "A survey on mobility-induced service migration in the fog, edge, and related computing paradigms," *ACM Comput. Surv.*, vol. 52, no. 5, pp. 1–33, 2019.
[3] T. Ouyang, Z. Zhou, and X. Chen, "Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 10, pp. 2333–2345, Oct. 2018.
[4] S. Wang, Y. Guo, N. Zhang, P. Yang, A. Zhou, and X. S. Shen, "Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 20, no. 3, pp. 939–951, Mar. 2021.
[5] C.-L. Wu, T.-C. Chiu, C.-Y. Wang, and A.-C. Pang, "Mobility-aware deep reinforcement learning with glimpse mobility prediction in edge computing," in *Proc. IEEE Int. Conf. Commun.*, 2020, pp. 1–7.
[6] S. Wang, R. Urgaonkar, M. Zafer, T. He, K. Chan, and K. K. Leung, "Dynamic service migration in mobile edge computing based on Markov decision process," *IEEE/ACM Trans. Netw.*, vol. 27, no. 3, pp. 1272–1288, Jun. 2019.
[7] X. Zhou, S. Ge, T. Qiu, K. Li, and M. Atiquzzaman, "Energy-efficient service migration for multi-user heterogeneous dense cellular networks," *IEEE Trans. Mobile Comput.* to be published, doi: 10.1109/TMC.2021.3087198.
[8] B. Wu, T. Chen, and X. Wang, "An mab approach for mec-centric task-offloading control in multi-rat hetnets," in *Proc. IEEE Int. Conf. Commun.*, 2020, pp. 1–6.
[9] Y. Sun, X. Guo, S. Zhou, Z. Jiang, X. Liu, and Z. Niu, "Learning-based task offloading for vehicular cloud computing systems," in *Proc. IEEE Int. Conf. Commun.*, 2018, pp. 1–7.
[10] Y. Sun, S. Zhou, and J. Xu, "EMM: Energy-aware mobility management for mobile edge computing in ultra dense networks," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 11, pp. 2637–2646, Nov. 2017.
[11] T. Ouyang, R. Li, X. Chen, Z. Zhou, and X. Tang, "Adaptive user-managed service placement for mobile edge computing: An online learning approach," in *Proc. IEEE Int. Conf. Comput. Commun.*, 2019, pp. 1468–1476.
[12] Q. Yuan, J. Li, H. Zhou, T. Lin, G. Luo, and X. Shen, "A joint service migration and mobility optimization approach for vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 9041–9052, Aug. 2020.
[13] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 3389–3396.
[14] D. Ye et al., "Mastering complex control in moba games with deep reinforcement learning," *Assoc. Adv. Artif. Intell.*, vol. 34, no. 4, pp. 6672–6679, 2020.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.

[15] S. Chinchali et al., "Cellular network traffic scheduling with deep reinforcement learning," in *Proc. Assoc. Advance. Artif. Intell.*, 2018, pp. 1–7.
[16] S. Wang, R. Urgaonkar, T. He, K. Chan, M. Zafer, and K. K. Leung, "Dynamic service placement for mobile micro-clouds with predicted future costs," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 4, pp. 1002–1016, Apr. 2017.
[17] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
[18] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1928–1937.
[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
[20] M. Igl, L. Zintgraf, T. A. Le, F. Wood, and S. Whiteson, "Deep variational reinforcement learning for pomdps," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 2117–2126.
[21] M. Watter, J. Springenberg, J. Boedecker, and M. Riedmiller, "Embed to control: A locally linear latent dynamics model for control from raw images," in *Proc. 28th Int. Conf. Neural Inf. Process. Syst.*, 2015, pp. 2746–2754.
[22] M. Zhang, S. Vikram, L. Smith, P. Abbeel, M. Johnson, and S. Levine, "SOLAR: Deep structured representations for model-based reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7444–7453.
[23] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, Cambridge, MA, USA: MIT Press, 2018.
[24] D. Ghosh and M. G. Bellemare, "Representations for stable off-policy reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 3556–3565.
[25] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
[26] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learn. Representations*, 2016.
[27] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations*, 2015.
[28] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015.
[29] L. Bracciale, M. Bonola, P. Loreti, G. Bianchi, R. Amici, and A. Rabuffi, "Crawdad dataset roma/taxi (v. 2014–07-17)," 2014. [Online]. Available: https://crawdad.org/roma/taxi/20140717
[30] M. Piorkowski, N. Sarafijanovic-Djukic, and M. Grossglauser, "Crawdad data set epfl/mobility (v.2009–02-24)," pp. 171–185, Feb. 2009.
[31] A. Narayanan et al., "A first look at commercial 5G performance on smartphones," in *Proc. Web Conf.*, 2020, pp. 894–905.
[32] L. Ma, S. Yi, N. Carter, and Q. Li, "Efficient live migration of edge services leveraging container layered storage," *IEEE Trans. Mobile Comput.*, vol. 18, no. 9, pp. 2020–2033, Sep. 2019.
[33] D. C. Nguyen, P. N. Pathirana, M. Ding, and A. Seneviratne, "Privacy-preserved task offloading in mobile blockchain with deep reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2536–2549, Dec. 2020.
[34] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.
[35] W. Zhan, C. Luo, G. Min, C. Wang, Q. Zhu, and H. Duan, "Mobility-aware multi-user offloading optimization for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 3341–3356, Mar. 2020.
[36] J. Kwak, Y. Kim, J. Lee, and S. Chong, "DREAM: Dynamic resource and task allocation for energy minimization in mobile cloud systems," *IEEE J. Sel. Areas Commun.*, vol. 33, no. 12, pp. 2510–2523, Dec. 2015.
[37] A. Nagabandi et al., "Learning to adapt in dynamic, real-world environments through meta-reinforcement learning," in *Proc. Int. Conf. Learn. Representations*, 2019.
[38] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surv. Tut.*, vol. 19, no. 4, pp. 2322–2358, Apr. 2017.
[39] C. Liu, F. Tang, K. Li, Z. Tang, and K. Li, "Distributed task migration optimization in MEC by extending multi-agent deep reinforcement learning approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1603–1614, Jul. 2021.
[40] J. Xu, X. Ma, A. Zhou, Q. Duan, and S. Wang, "Path selection for seamless service migration in vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 9, pp. 9040–9049, Sep. 2020.
[41] R. A. Addad, D. L. C. Dutra, T. Taleb, and H. Flinck, "Toward using reinforcement learning for trigger selection in network slice mobility," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 2241–2253, Jul. 2021.



Jin Wang received the BEng and MEng degrees in computer science from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2014 and 2017, respectively. He is currently working toward the PhD degree in computer science with the University of Exeter. His research interests include deep reinforcement learning, applied machine learning, cloud and edge computing, and computer system optimization.

Jia Hu received the BEng and MEng degrees in electronic engineering from the Huazhong University of Science and Technology, China, in 2006 and 2004, respectively, and the PhD degree in computer science from the University of Bradford, U.K., in 2010. He is a senior lecturer of computer science with the University of Exeter. His research interests include edge-cloud computing, resource optimization, applied machine learning, and network security.

Geyong Min (Member, IEEE) received the BSc degree in computer science from the Huazhong University of Science and Technology, China, in 1995, and the PhD degree in computing science from the University of Glasgow, United Kingdom, in 2003. He is a professor of high performance computing and networking with the Department of Computer Science within the College of Engineering, Mathematics and Physical Sciences with the University of Exeter, United Kingdom. His research interests include computer networks, wireless communications, parallel and distributed computing, ubiquitous computing, multimedia systems, modeling and performance engineering.

Qiang Ni (Senior Member, IEEE) is a professor with the School of Computing and Communications, Lancaster University, Lancaster, U.K. His research interests include the area of future generation communications and networking, including green communications and networking, millimeter-wave wireless communications, cognitive radio network systems, non-orthogonal multiple access (NOMA), heterogeneous networks, 5G and 6G, SDN, cloud networks, edge computing, dispersed computing, energy harvesting, wireless information and power transfer, IoTs, cyber physical systems, AI and machine learning, Big Data analytics, and vehicular networks. He has authored or co-authored 300+ papers in these areas. He was an IEEE 802.11 Wireless Standard Working Group Voting Member and a contributor to various IEEE wireless standards.

Tarek El-Ghazawi (Fellow, IEEE) received the PhD degree in electrical and computer engineering from New Mexico State University (NMSU). He is professor and chair of the Department of Electrical and Computer Engineering with the George Washington University, where he led the university-wide Strategic Academic Program in High-Performance Computing and the NSF Industry University CHREC Center. El-Ghazawi's research interests include high-performance computing, machine learning, computer and reconfigurable architectures, and distributed systems. El-Ghazawi has authored more than 300 refereed publications. His research has been funded extensively by US Government organizations including DARPA, NSF, AFOSR, NASA, DoD, and industry including Intel, AMD, HP, SGI. He is a Fellow of the IEEE, selected a Research Faculty Fellow of IBM/CAS and a Distinguished Visiting Fellow of the U.K. Royal Academy of Engineering. He was awarded the Alexander von Humboldt Research Award and the Alexander Schwarzkopf Prize for Technical Innovation. He served as U.S. Fulbright Scholar and as an IEEE Computer Society Distinguished Speaker.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 10:30:22 UTC from IEEE Xplore. Restrictions apply.